Now we can see some extra information: `class` and `object` give us information about the object the method `doB` was called on (although it retains the `function` key name); `type` concerns the context the call was made in. `->` means object context, and `::` would mean class (static) context. This might not mean much to you now; everything is explained in Chapter 6, "Object Fundamentals."

One final note to add: if the constant `DEBUG_BACKTRACE_IGNORE_ARGS` is passed as an argument to either of the backtrace functions, then the `args` index is omitted in the output; `DEBUG_BACKTRACE_PROVIDE_OBJECT` will omit the `object` index.

Summary

To misquote Douglas Adams: that about wraps it up for logic. The building blocks of control are easy enough to understand in isolation, but believe me, things can get very tortuous indeed. I recommend factoring out too many levels of nesting (either `if` statements, loops, or – horror of horrors – both embedded in each other). Beware of code that has closing braces like this:

```
                    }
                }
            }
        }
}
```

Something I like to call "climbing mountains" (if you turn the code on its side it looks like a mountain range). There's nothing wrong with refactoring monster loops and logic into their own functions (and your static analysis tools will be happier too). Over the years PHP has introduced syntactic sugar like ternary statements and null-coalesce to make things less onerous – make good use of them. Live long and prosper. 🖖